

LUIZA MANCHINI

The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query is a PL/pgSQL function named `bloquear_exclusao()` that checks if a book is available for deletion. The query is as follows:

```
9  CREATE OR REPLACE FUNCTION bloquear_exclusao()
10 RETURNS TRIGGER
11 LANGUAGE plpgsql
12 AS $$
13 BEGIN
14     IF OLD.quantidade > 0 THEN
15         RAISE EXCEPTION
16             'Não é possível excluir o livro "%", pois ainda existem % exemplar(es) disponível(is).',
17             OLD.titulo,
18             OLD.quantidade;
19     END IF;
20     RETURN OLD;
21 END;
22 $$;
23
24 ALTER TABLE livro
25 ADD COLUMN quantidade INTEGER DEFAULT 5;
26
27 CREATE TRIGGER trg_bloquear_exclusao
28 BEFORE DELETE
29 ON livro
30 FOR EACH ROW
31 EXECUTE FUNCTION bloquear_exclusao();
32
33 SELECT id_livro, titulo, quantidade
34 FROM livro
35 WHERE quantidade > 0
36 ORDER BY id_livro;
37
38 DELETE FROM livro
39 WHERE id_livro = 1;
```

The query execution resulted in an error:

```
Data Output Messages Notifications
ERROR: Não é possível excluir o livro "O Senhor dos Anéis", pois ainda existem 5 exemplar(es) disponível(is).
CONTEXT: PL/pgSQL function bloquear_exclusao() line 4 at RAISE
SQL state: P0001
```

Total rows: 4 Query complete 00:00:00.128

A notification box in the bottom right corner states: "You are currently running version 9.14 of pgAdmin 4, however the current version is 9.15. Please click [here](#) for more information."

The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query is a PL/pgSQL function named `log_exclusao()` that logs the deletion of a book and updates the quantity. The query is as follows:

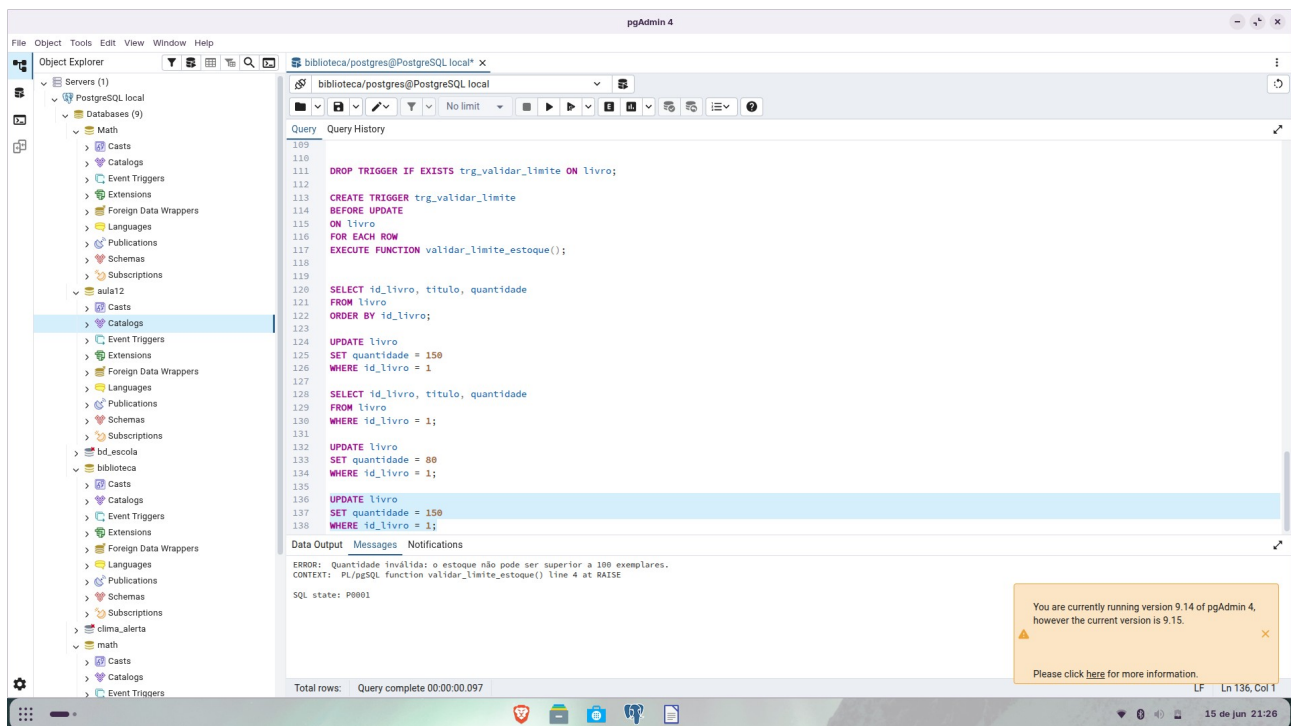
```
62
63 VALUES (
64     OLD.titulo,
65     CURRENT_TIMESTAMP,
66     'Livro removido do sistema'
67 );
68
69 RETURN OLD;
70 END;
71 $$;
72
73
74 DROP TRIGGER IF EXISTS trg_log_exclusao ON livro;
75
76 CREATE TRIGGER trg_log_exclusao
77 AFTER DELETE
78 ON livro
79 FOR EACH ROW
80 EXECUTE FUNCTION log_exclusao_livro();
81
82 SELECT id_livro, titulo, quantidade
83 FROM livro
84 ORDER BY id_livro;
85
86 UPDATE livro
87 SET quantidade = 0
88 WHERE id_livro = 5;
89
90 DELETE FROM livro
91 WHERE id_livro = 5;
```

The query execution was successful:

```
Data Output Messages Notifications
DELETE 0
Query returned successfully in 99 msec.
```

Total rows: Query complete 00:00:00.099

A notification box in the bottom right corner states: "You are currently running version 9.14 of pgAdmin 4, however the current version is 9.15. Please click [here](#) for more information."



--EX 1--

DROP TRIGGER IF EXISTS trg_bloquear_exclusao ON livro; DROP FUNCTION IF EXISTS bloquear_exclusao();

CREATE OR REPLACE FUNCTION bloquear_exclusao() RETURNS TRIGGER LANGUAGE plpgsql AS \$\$ BEGIN IF OLD.quantidade > 0 THEN RAISE EXCEPTION 'Não é possível excluir o livro "%", pois ainda existem % exemplar(es) disponível(is).', OLD.titulo, OLD.quantidade; END IF;

RETURN OLD;

END; \$\$;

ALTER TABLE livro ADD COLUMN quantidade INTEGER DEFAULT 5;

CREATE TRIGGER trg_bloquear_exclusao BEFORE DELETE ON livro FOR EACH ROW EXECUTE FUNCTION bloquear_exclusao();

SELECT id_livro, titulo, quantidade FROM livro WHERE quantidade > 0 ORDER BY id_livro;

DELETE FROM livro WHERE id_livro = 1;

--ex 02 --

DROP TABLE IF EXISTS log_livros_excluidos;

CREATE TABLE log_livros_excluidos (id_log SERIAL PRIMARY KEY, titulo VARCHAR(150), data_exclusao TIMESTAMP, mensagem VARCHAR(255));

```
CREATE OR REPLACE FUNCTION log_exclusao_livro() RETURNS TRIGGER LANGUAGE
plpgsql AS $$ BEGIN INSERT INTO log_livros_excluidos
( titulo, data_exclusao, mensagem ) VALUES ( OLD.titulo, CURRENT_TIMESTAMP, 'Livro
removido do sistema' );
```

```
RETURN OLD;
```

```
END; $$;
```

```
DROP TRIGGER IF EXISTS trg_log_exclusao ON livro;
```

```
CREATE TRIGGER trg_log_exclusao AFTER DELETE ON livro FOR EACH ROW EXECUTE
FUNCTION log_exclusao_livro();
```

```
SELECT id_livro, titulo, quantidade FROM livro ORDER BY id_livro;
```

```
UPDATE livro SET quantidade = 0 WHERE id_livro = 5;
```

```
DELETE FROM livro WHERE id_livro = 5;
```

```
--ex 03 --
```

```
CREATE OR REPLACE FUNCTION validar_limite_estoque() RETURNS
TRIGGER LANGUAGE plpgsql AS $$ BEGIN IF NEW.quantidade > 100 THEN RAISE
EXCEPTION 'Quantidade inválida: o estoque não pode ser superior a 100 exemplares.'; END IF;
```

```
RETURN NEW;
```

```
END; $$;
```

```
DROP TRIGGER IF EXISTS trg_validar_limite ON livro;
```

```
CREATE TRIGGER trg_validar_limite BEFORE UPDATE ON livro FOR EACH
ROW EXECUTE FUNCTION validar_limite_estoque();
```

```
SELECT id_livro, titulo, quantidade FROM livro ORDER BY id_livro;
```

```
UPDATE livro SET quantidade = 150 WHERE id_livro = 1
```

```
SELECT id_livro, titulo, quantidade FROM livro WHERE id_livro = 1;
```

```
UPDATE livro SET quantidade = 80 WHERE id_livro = 1;
```

```
UPDATE livro SET quantidade = 150 WHERE id_livro = 1;
```

Exercício 4 — Análise de Trigger BEFORE e AFTER

a) Diferença entre triggers BEFORE e AFTER

A trigger BEFORE é executada antes que uma operação de INSERT, UPDATE ou DELETE seja concluída no banco de dados. Ela pode analisar os dados, modificar valores utilizando a variável NEW ou impedir a operação por meio do comando RAISE EXCEPTION.

A trigger AFTER é executada somente depois que a operação foi realizada com sucesso. Ela é utilizada principalmente para registrar históricos, criar logs, atualizar tabelas auxiliares e executar ações que dependem da confirmação da alteração.

b) Qual deve ser usada para validação?

Para validações, deve-se utilizar a trigger BEFORE.

Isso acontece porque a validação precisa ser realizada antes de o dado ser gravado. Caso o valor seja inválido, a trigger pode interromper a operação com RAISE EXCEPTION, evitando que informações incorretas sejam salvas no banco.

Um exemplo é impedir que a quantidade de livros seja maior que 100.

c) Qual deve ser usada para auditoria?

Para auditoria, deve-se utilizar a trigger AFTER.

Ela é executada depois que a alteração foi concluída com sucesso. Assim, o sistema registra no log apenas operações que realmente ocorreram, como alterações de estoque, exclusões de livros ou mudanças de dados.

d) Por que a ordem de execução é importante?

A ordem é importante porque primeiro o banco deve validar os dados e somente depois registrar a alteração no histórico.

Por exemplo, ao atualizar a quantidade de um livro para 150:

1. A trigger BEFORE UPDATE verifica se o valor é permitido.
2. Como o valor é maior que 100, a operação é bloqueada.
3. O UPDATE não é realizado.
4. A trigger AFTER UPDATE não é executada.
5. Nenhum log de alteração é criado.

Isso evita que o banco registre em uma tabela de auditoria uma operação que foi recusada. Em bancos de dados reais, essa ordem ajuda a manter os dados e os históricos corretos e consistentes.

Exercício 5 — Reflexão sobre Integridade e Automação

a) Quais riscos essa decisão pode gerar?

Remover todas as triggers e manter as validações somente na aplicação pode gerar problemas de integridade e inconsistência dos dados.

Caso existam várias aplicações acessando o mesmo banco, cada uma poderá aplicar regras diferentes. Além disso, uma falha no frontend, no backend ou uma alteração feita diretamente por SQL poderá inserir, alterar ou excluir dados inválidos.

Outro risco é depender totalmente do funcionamento correto da aplicação. Se uma validação for esquecida durante uma atualização do sistema, o banco poderá receber informações incorretas.

b) Quais vantagens existem em manter regras diretamente no banco?

Manter algumas regras diretamente no banco garante que elas sejam aplicadas independentemente da aplicação utilizada.

As principais vantagens são:

- centralização das regras de integridade;
- proteção contra operações inválidas;
- padronização entre diferentes sistemas;
- criação automática de logs e históricos;
- maior segurança e consistência dos dados.

Assim, mesmo que uma aplicação apresente erro, o banco continua protegendo as informações.

c) Como triggers ajudam na integridade e consistência dos dados?

As triggers executam automaticamente uma função quando ocorre um evento como **INSERT**, **UPDATE** ou **DELETE**.

Elas ajudam na integridade porque podem validar os dados antes da gravação, impedir operações incorretas e registrar alterações realizadas. Dessa forma, as regras são aplicadas sempre da mesma maneira.

Por exemplo, uma trigger pode impedir que a quantidade de livros ultrapasse 100 exemplares ou impedir a exclusão de um livro que ainda possua unidades disponíveis.

d) Exemplo real de uso de triggers em sistemas corporativos

Em um sistema bancário, uma trigger pode registrar automaticamente todas as movimentações realizadas em uma conta.

Quando um saldo é alterado, a trigger pode gravar em uma tabela de auditoria:

- o saldo anterior;
- o saldo novo;
- a data e a hora;
- o tipo da operação;

- o usuário responsável.

Esse registro permite rastrear alterações, identificar falhas e aumentar a segurança das informações financeiras.